Connectors, Access and Repositories

A four-part lecture on how ThreadForge connects to AI models and tools, who may do what, and how the code and releases are protected. For learning, teaching and technical assessment.

01 AI & Connectors · slide and teaching notes

02 Access & Tokens · slide and teaching notes

03 How it connects · slide and teaching notes

04 Repository access · slide and teaching notes

05 Assessment: knowledge check (12 questions)

06 Assessment: practical labs and design proposal

07 Marking rubric

Learning outcomes

- Explain where an AI model runs and what each choice sends outside the computer.
- Give each person the least access they need, and issue, use and revoke a token safely.
- Trace one request from the browser to the database, naming the proof at each step.
- Describe who may change the repositories and the release path, and what is still to secure.

THREADFORGE 10.17 · LECTURE · PART 1

AI & Connectors

You choose where the model runs. Nothing outside your PC is used until the owner turns it on, tests it and saves it.

ThreadForge

1 Where the model runs

- **Ollama on this device:** a free local model at `127.0.0.1:11434`. No key, no per-token bill, nothing leaves the PC.
- **Ollama on another computer:** over HTTPS, for a stronger machine on your network.
- **Your own provider account:** OpenAI, Anthropic, Google Gemini, or any OpenAI-compatible service. Billed by the provider, with no ThreadForge markup.
- **Builder + reviewer:** an optional second model gives an independent second opinion.

2 MCP servers & plugins

- Listed in owner-written files on the server: an MCP servers file, and one folder per plugin with a `threadforge-plugin.json` manifest.
- **Everything starts disabled.** The owner enables each one and tests it.
- Only the tools the file allows are offered to an agent.
- Tool results are treated as **untrusted data**, never as instructions.

3 What uses the AI

- **AI review** of calculations and documents.
- **App builder & Studio:** plans an original app from plain words.
- **The Mentor's** later phases, and goal planning.
- **Engineering agents:** builder → reviewer → tests must pass → a **draft** pull request for a person to review. Agents never merge.
- The arithmetic is done by ThreadForge's own calculators, never by the model.

CHOOSE
Ollama on this device, another computer, or your provider

→

FILL
Address · model name (e.g. `qwen2.5-coder:7b`) · key only for a provider

→

TEST THE CONNECTION
One small request proves it answers

→

SAVE
Keys stay in protected files on the server, never returned to the browser

Local AI with Ollama	Provider accounts, plugins, MCP	Engineering agents	Custom tier
Professional and above	Industrial and above	Industrial and above	Pick only these desks: $19 + $15 per desk

Source: ThreadForge 10.17.0, AI & models desk and connectors module (business_connectors.py, licence.py)

PART 1 · TEACHING NOTES

AI & Connectors

Key points

- **Local first.** Ollama on the same PC needs no key, costs nothing per answer and sends nothing out.
- **Provider accounts** are billed by the provider. ThreadForge adds no markup.
- **Off by default.** MCP servers and plugins do nothing until the owner enables and tests each one.
- **Least tools.** An agent sees only the tools the owner's file allows.
- **Untrusted results.** What a tool returns is data, never an instruction to the agent.
- **People decide.** Agents open draft pull requests only. They never merge.

Demonstrate

1. Open **AI & models**. Choose **Ollama on this device**.
2. Address `http://127.0.0.1:11434/v1/chat/completions`, model `qwen2.5-coder:7b`, Key empty.
3. **Test the connection**, then **Save**.
4. Show the common mistake: a sentence typed in the Model box gives "model … is not downloaded".

Common misconceptions

- "The AI does the maths." No: ThreadForge's calculators do.
- "A key is needed for Ollama." No: only for provider accounts.

Discussion questions

1. A clinic must keep patient notes on site. Which model choice fits, and why?
2. Why should a tool's output never be treated as an instruction? Give an example of harm.
3. What is gained by a second "reviewer" model? What does it cost?
4. Why are engineering agents limited to draft pull requests?

Sources: business_connectors.py, licence.py, AGENT-BENCHMARK-2026-10.md

THREADFORGE 10.17 · LECTURE · PART 2

Access & Tokens

One owner, separate organizations, and a named token for every person, each with only the role it needs.

ThreadForge

1 Who holds what

Workspace owner: the workspace token. Engineering, host and provider settings, licence, backup and restore.

Organizations: each a separate database file, up to 100. Created by the owner only, with no public sign-up.

Named role tokens: bound to one organization. They cannot reach another organization or owner settings.

2 Four roles

ROLE	CAN CHANGE
Admin	Every business action, and tokens and invitations for that organization
Operator	Stock, orders, imports, sources, campaigns, projects, devices, due diligence
Finance	Payments, invoices, contacts, trade, the rate & policy log
Viewer	Nothing: read only. May still calculate and export.

Every role can read the organization's records and ask the Mentor.

3 Golden rules

- **One person, one named token.** Never share an admin token.
- A token is **shown once**. Store it in a protected file or password manager.
- Never paste a token into a chat, URL, screenshot or email.
- Engineering uses **separate credentials**: worker, sandbox, model key, GitHub token. Reusing one is refused.
- The licence key and the Android signing key never go onto a shared machine.

ISSUE
Name + role + expiry of 1–365 days

→

SHOWN ONCE
Only a keyed digest is stored

→

USE
Bound to one organization. Sign-ins last 12 hours, with forgery protection

→

REVOKE ANYTIME
Applies on the next request, with no restart

Invitations
One-time code, valid 7 days

Limits
100 active tokens and 50 pending invitations per organization

Audit
Every access change is recorded in the access ledger

Replace a token
Issue a new one, deliver it safely, then revoke the old one

Source: ThreadForge 10.17.0, People & access desk (tenancy.py) and TOKEN-ACCESS-POLICY.md

PART 2 · TEACHING NOTES

Access & Tokens

Key points

- **Least privilege:** each token has one role in one organization.
- **Shown once:** the server keeps only a keyed digest, so a lost token is replaced, not recovered.
- **Expiry** is 1 to 365 days, and **revocation** works on the next request.
- **Named tokens** make the access ledger meaningful: you can see who did what.
- **Separate duties:** worker, sandbox, model and GitHub credentials are all different, and reuse is refused.

Demonstrate

1. In **People & access**, issue a token named "Trainee · viewer" for 1 day.
2. Sign in with it in a private browser window.
3. Try to add a product: it is refused, "read-only".
4. Run a TradeMath calculation: it works, because calculating changes nothing.
5. Revoke the token, then refresh the window: the session ends.

Discussion questions

1. Why is sharing one admin token among staff worse than many named tokens?
2. Finance can change payments but not stock. What fraud does this separation help prevent?
3. Why store a digest instead of the token itself?
4. When would you choose a 7-day expiry over 365 days?

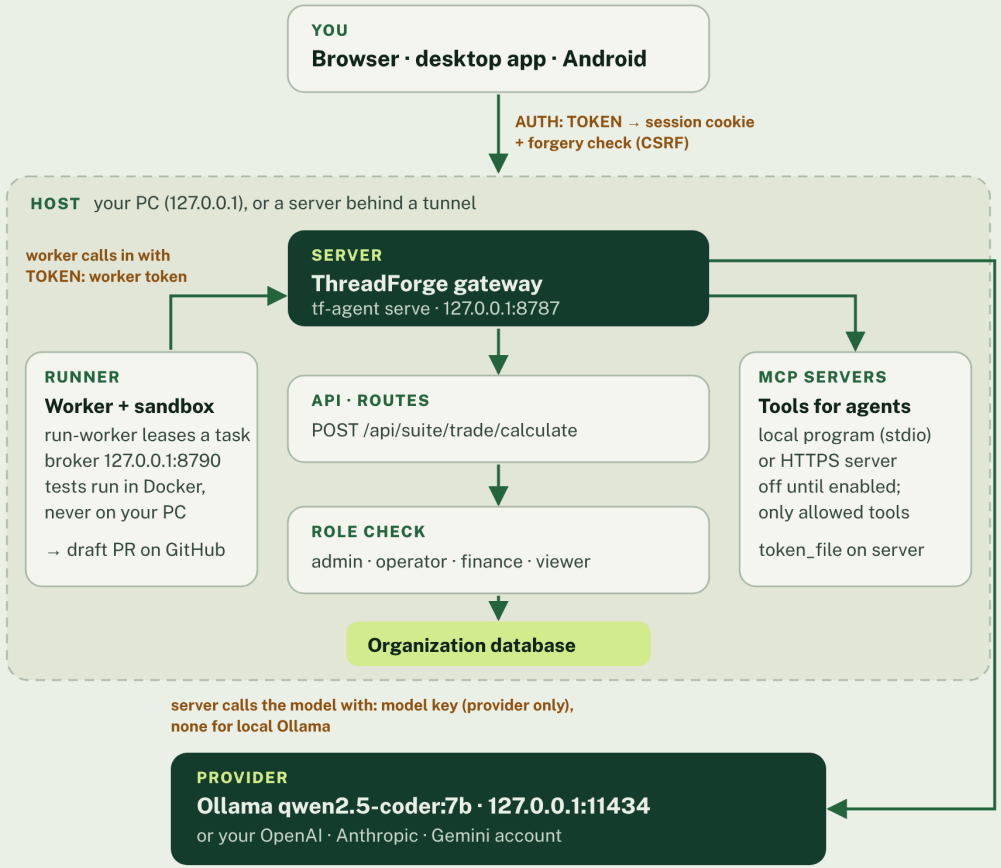
Sources: tenancy.py, server.py, TOKEN-ACCESS-POLICY.md

THREADFORGE 10.17 · LECTURE · PART 3

How it connects

Ten words, one map: who calls what, with which proof, and where each part runs.

ThreadForge



Arrows show who calls whom. Every arrow carries its own proof; no credential is reused.

ROLE
What a credential may change.
`operator → may record stock and orders, not payments`

ROUTE
One address on the server for one action.
`POST /api/suite/trade/calculate`

AUTH
How the server knows who is calling.
`Authorization: Bearer <token> · or session cookie + X-CSRF-Token`

MCP
Model Context Protocol: a standard way to give agents tools.
`"docs": transport http, allowed_tools ["search"]`

API
The set of routes that programs call.
`GET /api/suite/licence → tier, days left`

TOKEN
A secret that proves who you are. Shown once.
`"Ayesha · finance", 90 days, one organization`

SERVER
The program that answers the routes.
`tf-agent serve → 127.0.0.1:8787`

PROVIDER
Who runs the AI model.
`Ollama qwen2.5-coder:7b, or your Anthropic account`

RUNNER
Runs code and tests apart from the server.
`run-worker + broker :8790 (Docker); CI on GitHub's windows-2022`

HOST
The machine and address the server lives on.
`your PC 127.0.0.1, or a server reached through a Cloudflare tunnel`

Source: ThreadForge 10.17.0: server.py, tenancy.py, business_connectors.py, cli.py, examples/mcp-servers.example.json

PART 3 · TEACHING NOTES

How it connects

Key points

- **Follow the arrows:** the caller always presents its own proof.
- **Sign-in:** the browser trades a token for a session cookie, valid 12 hours. Every change also needs the CSRF code.
- **Two checks:** the server checks the route, then the role, before the database is touched.
- **The worker calls in;** the server never reaches out to it. Tests run inside Docker.
- **The host** can be a PC on 127.0.0.1, or a server reached through a tunnel.

Trace exercise

Walk the class through one request:

1. An operator saves a stock movement in the browser.
2. **Auth:** session cookie and CSRF code are checked.
3. **Route:** the request goes to an items route under `/api/suite/`.
4. **Role:** operator may change items: allowed.
5. **Database:** the organization's own file is updated and the change is recorded.

Repeat it as **finance**, and stop at the role check with "outside the credential's organization role".

Discussion questions

1. Why does the worker call the server instead of the server calling the worker?
2. What could go wrong if tests ran directly on the host, outside Docker?
3. Which arrow carries no secret at all when Ollama runs locally?
4. Where would you add a firewall rule, and what would it allow?

Sources: server.py, tenancy.py, cli.py, examples/mcp-servers.example.json

THREADFORGE 10.17 · LECTURE · PART 4

Repository access

Two repositories, a short list of who may touch them, and the road to a locked-down release path.

ThreadForge

PRIVATE

ThreadForge-Inc/SO

- **Source code,** tests and the build pipelines.
- **CI on GitHub's machines:** Windows, Android, macOS, MSIX, VS Code.
- **Verified releases**, e.g. `v10.17.0-dev`, pinned to exact runs and SHA-256 sums.
- Old name `kingswomen212-create/SO` still redirects.

PUBLIC

threadforge-website

- **The live site** threadforgeofficial.com, served by GitHub Pages.
- **Public downloads:** 11 files per release, copied byte for byte from SO.
- **Source code never goes here.** The source ZIP, wheel and sdist stay private.

Who may do what

WHO	ACCESS, AND WITH WHAT
Owner account	Admin on both. Approves merges and Android signing
Pull-request CI	Read only. Signs Android with a throwaway key
Signing run on main	The permanent key, from `android-release` only
Release workflow	Write releases on SO. `WEB_RELEASE_TOKEN` for the website release
Engineering agents	A fine-grained token in a file. Allowlisted repositories only, draft PRs only

DONE

- ✓ Moved to the **ThreadForge-Inc** organization
- ✓ PR builds can no longer reach the Android key (#68)
- ✓ `android-release` environment created, keystore added
- ✓ Releases check every byte against their CI runs
- ✓ The site checker blocks wrong versions and withdrawn claims (#67)

YOUR STEPS NOW

- → Add yourself as **required reviewer** on `android-release`, limited to `main`
- → Delete the 3 repository-level keystore secrets
- → Protect `main`: require a PR and passing checks
- → For agents: a **fine-grained token** for SO only, saved in a file
- → No self-hosted runner on your personal PC

NEXT PR (CLAUDE)

- ○ Point 4 signing and verify workflows at `ThreadForge-Inc/SO`
- ○ Update `SETUP.md` and the token guide to the new name
- ○ Re-run the signing configuration check

LATER

- ▪ Windows publisher signing through Azure, in its own protected environment
- ▪ Secret scanning and dependency alerts switched on
- ▪ Review who has access every quarter

Source: this session's merged PRs #67–#73, the release workflows and TOKEN-ACCESS-POLICY.md

Repository access

Key points

- **Private source, public downloads:** only the 11 release files reach the public website repository.
- **The signing key** is reachable only from an approved run on `main`. Pull-request builds get a throwaway key.
- **Releases are pinned:** exact CI runs, exact artifact digests, exact SHA-256 values. Anything else is refused.
- **A roadmap is honest** when it separates what is done from what is still to do.

Case study: 10.17.0 release

1. Version bump merged, then both builds ran from one `main` commit.
2. GitHub returned an incomplete list of job steps. The release refused to publish rather than guess.
3. The fix read the job log, with the same strict checks. A GitHub 502 error was retried once.
4. Published only after every byte matched; then copied to the website.

Lesson: a safe pipeline fails closed.

Discussion questions

1. Why should a personal PC not run a self-hosted CI runner for a repository others can push to?
2. What does "fail closed" mean, and where did you see it in this case?
3. Who should be the required reviewer for releases in a two-person company?
4. What is lost if source archives are published by mistake?

Sources: release workflows, merged PRs #67 to #73

ASSESSMENT 1 · 20 MINUTES · INDIVIDUAL

Knowledge check

1. Which model choice sends nothing outside the PC?
A. Anthropic account · B. Ollama on another computer · C. Ollama on this device · D. Gemini

2. A newly listed MCP server is…
A. enabled with all tools · B. disabled until the owner enables and tests it · C. enabled with read-only tools · D. hidden

3. How must an agent treat what a tool returns?
A. as instructions · B. as untrusted data · C. as the owner's request · D. as a licence

4. Which licence tier first includes local AI with Ollama?
A. Essentials · B. Professional · C. Industrial · D. Industrial Park only

5. What can an engineering agent do with its finished work?
A. merge to main · B. publish a release · C. open a draft pull request · D. delete the branch

6. A viewer token tries to add a product. The server answers…
A. 200 OK · B. 403, read-only · C. 404 · D. 500

7. What range of expiry can an organization token have?
A. 1 hour to 1 day · B. 1 to 30 days · C. 1 to 365 days · D. never expires

8. A finance token may change…
A. stock items · B. payments and invoices · C. other organizations · D. AI provider settings

9. How long does a signed-in browser session last?
A. 30 minutes · B. 12 hours · C. 7 days · D. until logout only

10. In the map, who starts the connection between the worker and the server?
A. the server · B. the worker, with its own worker token · C. the provider · D. the browser

11. Which builds may use the permanent Android signing key?
A. every pull request · B. any branch · C. an approved run on main · D. a self-hosted runner

12. Which files are copied to the public website repository?
A. everything · B. the source ZIP · C. the 11 public release files only · D. none

ASSESSMENT 2 · PRACTICAL · PAIRS OR INDIVIDUAL

Labs and design proposal

LAB A Connect a local model

Task: install Ollama, pull `qwen2.5-coder:7b`, connect it on **AI & models** with no key, and test it.

Evidence: a screenshot of the passed test, and one sentence on what left the PC.

Pass: the test passes, the Key box is empty, and the answer is "nothing".

LAB B Least privilege

Task: issue a finance token for 1 day. Show that it can record a payment and cannot add stock. Then revoke it.

Evidence: the two results (one allowed, one refused with its message) and the revocation in the ledger.

LAB C Trace a request

Task: using the Part 3 map, trace three requests and name the layer that answers each one:

1. A viewer runs a CBM calculation.
2. An operator records a payment.
3. A request arrives with no cookie and no token.

Evidence: a three-row table: request, layer, result and message.

PROPOSAL Design access for a firm

Brief: "Example Traders" has an owner, a store manager, an accountant and a seasonal helper. They want local AI only, and one engineer contributes code.

Proposal: what to submit

1. **Access table:** each person, their role, the expiry, and the reason.
2. **AI plan:** where the model runs, what is sent out, and the licence tier needed.
3. **Request map:** the Part 3 map redrawn for this firm, with each arrow's proof.
4. **Repository plan:** who may merge, who approves releases, and how the engineer's token is limited.
5. **Risks:** the three biggest risks, and how each is reduced.
6. **Offboarding:** the steps when the seasonal helper leaves.

Length: 2 to 4 pages and one diagram. Marked with the rubric on the next page.

ASSESSMENT 3 · MARKING

Rubric for the design proposal

CRITERION	WEIGHT	EXCELLENT (4)	GOOD (3)	DEVELOPING (2)	MISSING (0–1)
Least privilege	25%	Every person has the narrowest role, with an expiry and a reason	Roles fit, but one is broader than needed	A shared or admin token is given without need	No roles, or everyone is admin
Data and AI choices	20%	Model location matches the firm's needs, and data leaving the site is named precisely	A sound choice, but data flow only partly explained	A choice is made without reasons	No AI plan
Request map	20%	Correct arrow directions, and every arrow's proof named	One arrow or proof wrong	Several errors in direction or proof	No map
Repository and release safety	15%	Required reviewer, protected main, scoped engineer token, no personal runner	Three of the four	One or two	None
Risks and offboarding	10%	Real risks with concrete reductions, and offboarding revokes and replaces access	Risks named, reductions vague	Generic risks only	Missing
Clarity	10%	Plain, accurate and complete; a reader could act on it	Clear with small gaps	Hard to follow	Unreadable

Grade bands: 85% and above distinction · 70–84% merit · 50–69% pass · below 50% resubmit. Labs A to C are pass or fail on their stated evidence.